# Kiến trúc máy tính RISC-V 32-bit (RV32I) trên Logisim Evolution

Tài liệu học từng bước: từng khối, bên trong có gì, vì sao

Nhóm bài tập lớn — bản LaTeX tiếng Việt

## Mục lục

**1 Mục tiêu và cách dùng tài liệu** **3**

**2 Giai đoạn 0 — Công cụ và bức tranh lớn (buổi 1)** **3**

**3 Giai đoạn 1 — Nền tảng (buổi 2–3)** **4**

3.1 CPU chạy chương trình như thế nào . . . 4

3.2 Định dạng lệnh RISC-V . . . 4

**4 Giai đoạn 2 — Giải mã và điều khiển (buổi 4)** **5**

4.1 Khối `InstructionDecode` . . . 5

4.2 Khối `MainControl` . . . 5

4.3 Khối `ImmGen` . . . 6

**5 Giai đoạn 3 — Thanh ghi và số học (buổi 5)** **7**

5.1 Khối `RegisterFile` . . . 7

5.2 Khối `ALUControl` . . . 7

5.3 Khối `ALU` . . . 8

5.4 Khối `BranchUnit` . . . 9

**6 Giai đoạn 4 — Bộ nhớ và CPU single-cycle (buổi 6–7)** **9**

6.1 Khối `Memory` cùng `LoadExtract` và `StoreMerge` . . . 9

6.2 CPU single-cycle `main` . . . 10

**7 Giai đoạn 5 — Pipeline (buổi 8–10)** **10**

7.1 Các tầng, thanh ghi pipeline, thời điểm ghi . . . 10

7.2 Khối `ForwardingUnit` . . . 12

7.3 Khối `HazardUnit` . . . 12

7.4 Hazard điều khiển và hiệu năng . . . 13

**8 Giai đoạn 6 — Chương trình, kiểm thử, công cụ (buổi 11–12)** **14**

8.1 Chương trình và công cụ . . . 14

8.2 Ba mức kiểm thử . . . 14

8.3 Lỗi đã tìm thấy và bài học . . . 16

**9 Đọc sâu file mạch `RiscVComputer.circ`** **16**
9.1 Số linh kiện trong từng circuit . . . 17
9.2 Phương trình đầy đủ của `MainControl` . . . 17
9.3 Bảng mã trong `ALUControl` . . . 18
9.4 Mười multiplexer của `Pipeline` . . . 18
9.5 Đóng gói từ điều khiển . . . 19
9.6 Xung nhịp . . . 19
9.7 Những điểm mạch *không* làm (nên biết khi bị hỏi) . . . 19

**10 Demo từng bước** **20**
10.1 Chương trình `demo.s` . . . 20
10.2 Cách đọc một tick . . . 20
10.3 Một lệnh đi qua CPU, theo từng nhóm . . . 20
10.4 Cả 40 tick của `demo.s` (single-cycle) . . . 21
10.5 Các giá trị probe cần xem (single-cycle) . . . 22

**11 Chia việc cho nhóm ba người** **22**

**12 Tự đánh giá** **22**

# 1 Mục tiêu và cách dùng tài liệu

Sau khi học hết tài liệu này, bạn có thể giải thích **mọi circuit của dự án, bên trong có gì và vì sao**, từ một cổng logic đến CPU pipeline 5 tầng, và chứng minh bằng kiểm thử rằng nó chạy đúng.

**Dự án gồm:** hai CPU dùng chung các khối: `main` (single-cycle) và `Pipeline` (5 tầng IF, ID, EX, MEM, WB); 14 circuit; bộ nhớ kiểu Harvard (ROM lệnh 256 từ, RAM dữ liệu 256 từ); assembler, bộ mô phỏng tham chiếu và bộ kiểm thử tự động.

**Cách học:** làm lần lượt các bước. Mỗi bước gồm *đọc*, *nhìn* (mở circuit trong Logisim), *làm* (một thí nghiệm nhỏ) và *kiểm tra* (trả lời không nhìn tài liệu). Chưa trả lời được thì lặp lại bước đó.

| Giai đoạn | Buổi | Chủ đề | Circuit |
|---|---|---|---|
| 0 | 1 | Công cụ và bức tranh lớn | (nhìn từ ngoài) |
| 1 | 2–3 | Nền tảng: CPU chạy chương trình, định dạng lệnh | — |
| 2 | 4 | Giải mã và điều khiển | `InstructionDecode`, `MainControl`, `ImmGen` |
| 3 | 5 | Thanh ghi và số học | `RegisterFile`, `ALUControl`, `ALU`, `BranchUnit` |
| 4 | 6–7 | Bộ nhớ và CPU single-cycle | `Memory`, `LoadExtract`, `StoreMerge`, `main` |
| 5 | 8–10 | Pipeline | `Pipeline`, `ForwardingUnit`, `HazardUnit` |
| 6 | 11–12 | Chương trình, kiểm thử, công cụ, demo | assembler, simulator, test |

Bảng 1: Lộ trình 12 buổi (mỗi buổi khoảng 90 phút, tổng cộng khoảng 18 giờ).

Nếu ba người chia việc: mỗi người học sâu một phần (xem bảng ở phần 11) nhưng cả ba cùng làm giai đoạn 0, 1 và 6.

# 2 Giai đoạn 0 — Công cụ và bức tranh lớn (buổi 1)

### Bước 0.1: mở dự án

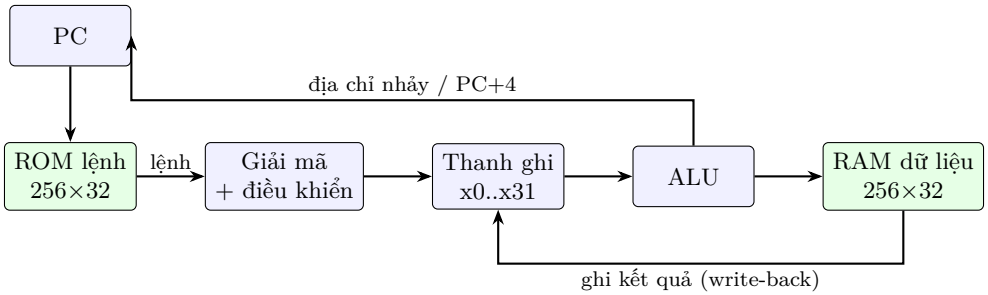
1. Cần Java 21. Mở dự án bằng lệnh:
   `java -jar logisim-evolution-4.1.0-all.jar RiscVComputer.circ`
2. Trong cây bên trái có 14 circuit. Chia thành ba nhóm:
   - hai CPU: `main`, `Pipeline`;
   - các khối datapath: `InstructionDecode`, `MainControl`, `ImmGen`, `RegisterFile`, `ALUControl`, `ALU`, `BranchUnit`, `Memory`, `LoadExtract`, `StoreMerge`;
   - các khối chỉ có trong pipeline: `ForwardingUnit`, `HazardUnit`.

### Bước 0.2: các thao tác Logisim cần biết

| Phím / thao tác | Tác dụng |
|---|---|
| Ctrl+R | đặt lại mô phỏng (PC và mọi thanh ghi về 0) |
| Ctrl+F9 | một chu kỳ xung nhịp đầy đủ (một tick) |
| Tab State | hiện `PC` và `x0..x31` trực tiếp |
| Tunnel (đường hầm) | các tunnel cùng tên là **cùng một dây** |
| Hàng probe cuối mạch | các giá trị cần theo dõi khi demo |

### Bước 0.3: bức tranh lớn

Hình 1 cho thấy ý tưởng chung: CPU lấy lệnh từ ROM, giải mã, thực thi bằng ALU, truy cập RAM và ghi kết quả vào thanh ghi.




Hình 1: Bức tranh lớn: nạp lệnh → giải mã → thanh ghi → ALU → bộ nhớ → ghi lại.

**Câu hỏi tự kiểm tra**

- `main` khác `Pipeline` ở điểm nào?
- Vì sao có hai bộ nhớ (ROM lệnh, RAM dữ liệu)?
- Tunnel dùng để làm gì?

# 3 Giai đoạn 1 — Nền tảng (buổi 2–3)

## 3.1 CPU chạy chương trình như thế nào

1. **Nạp lệnh (fetch):** đọc lệnh ở địa chỉ trong `PC`.
2. **Giải mã (decode):** tách trường lệnh, đọc thanh ghi, sinh tín hiệu điều khiển.
3. **Thực thi (execute):** ALU tính; quyết định rẽ nhánh.
4. **Bộ nhớ (memory):** load / store.
5. **Ghi lại (write-back):** đưa kết quả vào thanh ghi `rd`; `PC` chuyển sang lệnh kế tiếp.

Các ý quan trọng: trạng thái chỉ đổi ở **cạnh xung nhịp**; mạch tổ hợp khác mạch tuần tự; số âm dùng bù hai (ví dụ $-1$ = `0xFFFFFFFF`); mọi giá trị trong tài liệu viết ở hệ 16 (hex) nếu có tiền tố `0x`.

**Thí nghiệm**

Chạy tay `programs/demo.s` 3 vòng lặp trên giấy (PC, x10, x11) rồi so với bảng 40 tick ở phần 10.

## 3.2 Định dạng lệnh RISC-V

Có sáu định dạng R, I, S, B, U, J. Các trường nằm **cố định** ở cùng vị trí bit nên giải mã chỉ là đi dây.

| Kiểu | 31–25 | 24–20 | 19–15 | 14–12 | 11–7 | 6–0 |
|---|---|---|---|---|---|---|
| R | funct7 | rs2 | rs1 | funct3 | rd | opcode |
| I | imm[11:0] | | rs1 | funct3 | rd | opcode |
| S | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode |
| B | imm[12\|10:5] | rs2 | rs1 | funct3 | imm[4:1\|11] | opcode |
| U | imm[31:12] | | | | rd | opcode |
| J | imm[20\|10:1\|11\|19:12] | | | | rd | opcode |

**Chín opcode đang dùng:** `0x33` (R), `0x13` (I-ALU), `0x03` (load), `0x23` (store), `0x63` (branch), `0x37` (LUI), `0x17` (AUIPC), `0x6F` (JAL), `0x67` (JALR).

**Thí nghiệm**

Chạy `python tools/rv.py asm programs/demo.s -l`, rồi giải mã tay ba từ lệnh `00b50533`, `fff58593`, `fe059ce3`: viết các trường dạng nhị phân, nêu định dạng và tên lệnh.

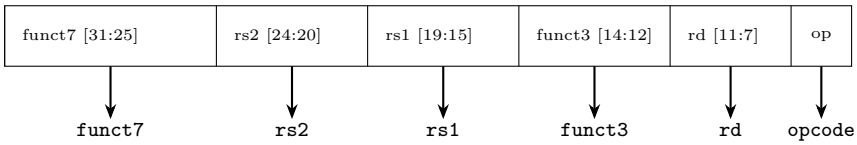
**Câu hỏi tự kiểm tra**

- Định dạng nào không có `rd`?
- Vì sao hằng số kiểu B và J thiếu bit 0?
- `funct7[5]` làm thay đổi điều gì?

# 4 Giai đoạn 2 — Giải mã và điều khiển (buổi 4)

## 4.1 Khối `InstructionDecode`

- **Mục đích:** cắt lệnh 32 bit thành các trường.
- **Vào / ra:** vào `Instruction`; ra `opcode`(7), `rd`(5), `funct3`(3), `rs1`(5), `rs2`(5), `funct7`(7).
- **Bên trong:** đúng một *splitter* (32 → 6 phần). Không có cổng logic.
- **Vì sao:** RISC-V đặt số hiệu thanh ghi luôn ở cùng chỗ, nên có thể đọc thanh ghi *trong khi* còn đang giải mã opcode.




Hình 2: `InstructionDecode`: chỉ là đi dây từ các trường bit cố định.

**Thí nghiệm**

Gán `Instruction` = `00b50533` rồi đọc sáu ngõ ra (`opcode` = `0x33`, `rd` = 10, `rs1` = 10, `rs2` = 11).

**Câu hỏi tự kiểm tra**

Các bit nào tạo `rs2`? Vì sao khối này không có cổng logic nào?

## 4.2 Khối `MainControl`

- **Mục đích:** từ `opcode` sinh tất cả tín hiệu điều khiển của datapath.
- **Vào / ra:** vào `opcode`(7); ra `RegWrite`, `ALUSrc`, `MemWrite`, `MemRead`, `IsBranch`, `IsJAL`, `IsJALR`, `ALUOp`(2), `ImmSel`(3), `ASel`(2), `WBSel`(2).
- **Bên trong:** chín *bộ so sánh* 7 bit (mỗi opcode một cái, kèm *hằng số* 7 bit); các *cổng OR* (tối đa 3 ngõ vào) cho biết lớp lệnh nào bật tín hiệu nào; các *splitter* ghép bit đơn thành bus 2–3 bit.

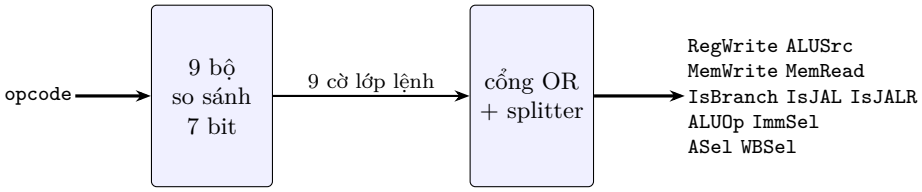
**Các phương trình ghép bus** (viết $\{a, b, \dots\}$: phần tử đầu là bit cao nhất):

$$\begin{aligned}
\texttt{ALUOp} &= \{\texttt{IsR} \vee \texttt{IsI},\ \texttt{IsI} \vee \texttt{IsBranch}\} \\
\texttt{ImmSel} &= \{\texttt{IsJAL},\ \texttt{IsBranch} \vee \texttt{IsLui} \vee \texttt{IsAuipc},\ \texttt{IsStore} \vee \texttt{IsLui} \vee \texttt{IsAuipc}\} \\
\texttt{ASel} &= \{\texttt{IsLui},\ \texttt{IsAuipc}\} \\
\texttt{WBSel} &= \{\texttt{IsJAL} \vee \texttt{IsJALR},\ \texttt{IsLoad}\}
\end{aligned}$$

Ý nghĩa các mã:

- `ALUOp`: 00 cộng, 01 trừ, 10 tra bảng kiểu R, 11 tra bảng kiểu I;
- `ImmSel`: 000 I, 001 S, 010 B, 011 U, 100 J;
- `ASel` (toán hạng A): 00 `rs1`, 01 `PC` (AUIPC), 10 hằng 0 (LUI);
- `WBSel` (dữ liệu ghi lại): 00 kết quả ALU, 01 dữ liệu bộ nhớ, 10 `PC+4` (JAL, JALR).

Opcode lạ làm mọi tín hiệu bằng 0 (một lệnh NOP vô hại).




Hình 3: `MainControl`: so sánh opcode → cờ lớp lệnh → OR → tín hiệu điều khiển.

**Thí nghiệm**

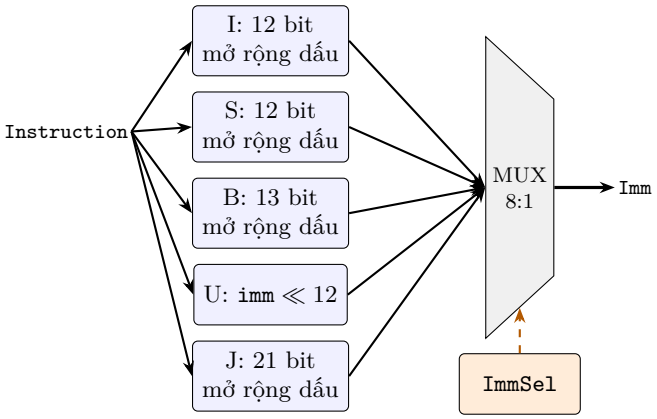
Điền bảng tín hiệu điều khiển trên giấy cho `lw`, `sw`, `beq`, `lui`, `jal`, rồi đối chiếu với `tests/maincontrol.vec`.

**Câu hỏi tự kiểm tra**

Với `sw`, những tín hiệu nào bằng 1? `ASel`=10 nghĩa là gì và dùng cho lệnh nào? Chuyện gì xảy ra với opcode 0?

### 4.3 Khối `ImmGen`

- **Mục đích:** dựng lại hằng số 32 bit theo định dạng do `ImmSel` chọn.
- **Vào / ra:** vào `Instruction`, `ImmSel`(3); ra `Imm`(32).
- **Bên trong:** 9 *splitter* (cắt các trường rải rác và ghép đúng thứ tự, trong đó một splitter 32 bit ghép 12 bit 0 với `instr[31:12]` cho kiểu U); 4 *bộ mở rộng bit* mở rộng dấu cho I, S, B, J (12, 12, 13, 21 bit → 32); với kiểu U là hằng số 12 bit 0 và splitter ghép (`imm`≪ 12); hai hằng số 1 bit 0 cho bit 0 luôn bằng 0 của B và J; một *multiplexer* 8-ra-1 (dùng 5 ngõ) chọn theo `ImmSel` (0 I, 1 S, 2 B, 3 U, 4 J).
- **Vì sao:** các bit hằng số bị xáo trộn có chủ đích để đa số bit nằm cùng vị trí giữa các định dạng; splitter xếp lại; mở rộng dấu giữ số âm vẫn âm.




Hình 4: `ImmGen`: năm cách dựng hằng số, multiplexer chọn theo `ImmSel`.

**Thí nghiệm**

Giải mã `fe059ce3` (kiểu B): hằng số phải là `0xfffffff8` = −8. Kiểm tra trong `tests/immgen.vec`.

**Câu hỏi tự kiểm tra**

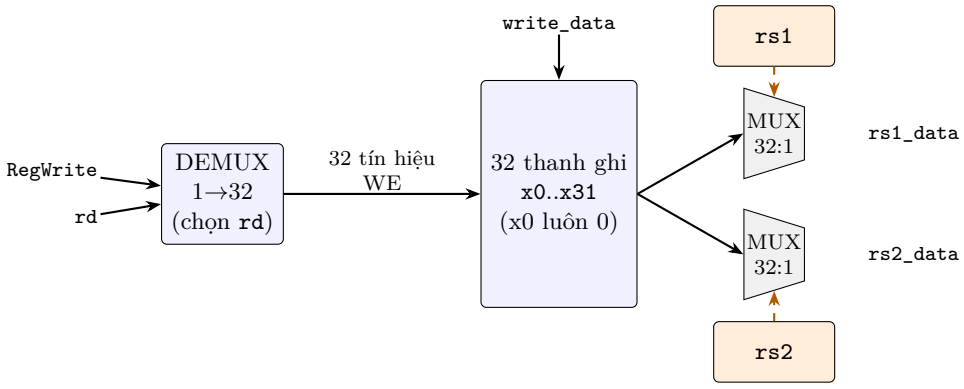
Vì sao độ lệch của B luôn chẵn? Bit dấu lấy từ đâu ở mỗi định dạng?

**Bài kiểm tra cuối giai đoạn:** từ một từ lệnh, nêu định dạng, ghi `rd`, `rs1`, `rs2`, hằng số và các tín hiệu điều khiển mà không nhìn tài liệu.

# 5 Giai đoạn 3 — Thanh ghi và số học (buổi 5)

## 5.1 Khối `RegisterFile`

- **Mục đích:** 32 thanh ghi × 32 bit, mỗi chu kỳ đọc hai và ghi một.
- **Vào / ra:** vào `write_data`, `CLK`, `write` (`RegWrite`), `rd`, `rs1`, `rs2`; ra `rs1_data`, `rs2_data`.
- **Bên trong:** 32 *thanh ghi* (nhãn `x0..x31`, sườn lên); một *demultiplexer* (chọn 5 bit, bit dữ liệu = `RegWrite`) dẫn tín hiệu cho phép ghi tới đúng thanh ghi `rd`; hai *multiplexer* 32-ra-1 làm cổng đọc (bất đồng bộ); hằng số 0 nối vào `x0`.
- **Cách hoạt động:** ghi: chỉ thanh ghi số `rd` có WE = 1; đọc: multiplexer luôn hiện thanh ghi do `rs1`/`rs2` chọn. `x0` chỉ nạp được 0 nên luôn bằng 0.
- **Vì sao:** đọc bất đồng bộ để CPU single-cycle có toán hạng ngay trong chu kỳ; trong pipeline khối này dùng **xung nhịp đảo** (ghi ở sườn xuống).




Hình 5: `RegisterFile`: demux dẫn WE tới `rd`; hai multiplexer là hai cổng đọc.

**Thí nghiệm**

Trong `main`, bấm 3 tick của demo rồi mở `RegisterFile`: tìm thanh ghi nào vừa đổi và đầu ra demux nào bằng 1.

**Câu hỏi tự kiểm tra**

Điều gì xảy ra khi `rd` = 0 và `RegWrite` = 1? Vì sao đọc bất đồng bộ còn ghi theo xung nhịp?

## 5.2 Khối `ALUControl`

- **Mục đích:** biến `ALUOp`, `funct3`, `funct7` thành mã 4 bit `ALUCtrl`.
- **Vào / ra:** vào `funct7`(7), `ALUOp`(2), `funct3`(3); ra `ALUCtrl`(4).

- **Bên trong:** 19 hằng số 4 bit (các mã: 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 SLL, 6 SRL, 7 SRA, 8 SLT, 9 SLTU) và 5 multiplexer: hai 8-ra-1 (bảng kiểu R và kiểu I, chọn theo `funct3`), hai 2-ra-1 (ADD/SUB và SRL/SRA, chọn theo `funct7[5]`), một 4-ra-1 (chọn theo `ALUOp`: ép ADD, ép SUB, bảng R, bảng I).
- **Vì sao:** một bảng mã = mỗi dòng một hằng số, multiplexer chọn dòng; điều khiển hai tầng (`MainControl` → `ALUOp` → `ALUControl`) giữ các bảng nhỏ.

| `ALUOp` | Nhóm lệnh | `ALUCtrl` | Ghi chú |
|---|---|---|---|
| 00 | load, store, LUI, AUIPC, JALR | ADD | luôn cộng |
| 01 | branch | SUB | so sánh bằng phép trừ |
| 10 | kiểu R | theo `funct3`, `funct7[5]` | ADD/SUB, SRL/SRA phân biệt bằng `funct7[5]` |
| 11 | kiểu I | theo `funct3` | chỉ lệnh dịch xét `funct7[5]` |

Bảng 2: Hai tầng điều khiển ALU.

**Thí nghiệm**

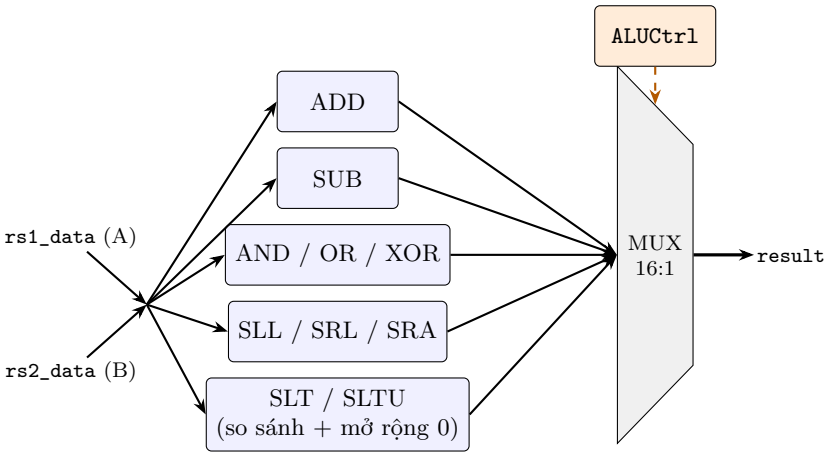
Viết trên giấy mã của `sub`, `sra`, `srai`, `slti`, `lw`, `beq` rồi đối chiếu `tests/alucontrol.vec`.

**Câu hỏi tự kiểm tra**

Vì sao `addi` bỏ qua `funct7` còn `srai` thì không? Load và branch dùng `ALUOp` nào, vì sao?

### 5.3 Khối `ALU`

- **Mục đích:** tính kết quả theo mã `ALUCtrl`.
- **Vào / ra:** vào `rs1_data`, `rs2_data`, `ALU_Ctrl`; ra `result`.
- **Bên trong:** bộ cộng, bộ trừ, cổng AND/OR/XOR 32 bit, ba bộ dịch (logic trái, logic phải, số học phải; lượng dịch `B[4:0]`), bộ so sánh có dấu và không dấu, mỗi bộ theo sau là bộ mở rộng 0 từ 1 lên 32 bit (SLT/SLTU trả 0 hoặc 1), một multiplexer 16-ra-1 (dùng 10 ngõ).
- **Cách hoạt động:** mọi phép tính chạy song song, multiplexer chọn một kết quả theo `ALUCtrl`.




Hình 6: `ALU`: tính song song mọi phép, multiplexer chọn theo `ALUCtrl`.

**Thí nghiệm**

Đặt A = −1, B = 1 với `ALUCtrl`=8 rồi 9: SLT cho 1, SLTU cho 0. Đối chiếu `tests/alu.vec`.

**Câu hỏi tự kiểm tra**

Vì sao kết quả SLT phải mở rộng 0? Bộ phận nào giúp `sra` giữ dấu?

### 5.4 Khối `BranchUnit`

- **Mục đích:** quyết định branch có được rẽ hay không.
- **Vào / ra:** vào `rs1_data`, `rs2_data`, `IsBranch`, `funct3`; ra `branch_taken`.
- **Bên trong:** một bộ so sánh không dấu và một có dấu 32 bit (cho $=$, $<_u$, $<_s$); 3 cổng NOT (cho $\neq$, $\geq_u$, $\geq_s$); multiplexer 8-ra-1 một bit chọn theo `funct3` (000 beq, 001 bne, 100 blt, 101 bge, 110 bltu, 111 bgeu; 010 và 011 cho 0); cổng AND với `IsBranch`.
- **Vì sao:** sáu điều kiện là ba phép so sánh và phủ định của chúng; AND với `IsBranch` đảm bảo lệnh khác không bao giờ rẽ.

**Câu hỏi tự kiểm tra**

`blt` và `bltu` khác nhau trong mạch ở chỗ nào?

**Bài kiểm tra cuối giai đoạn:** tính tay `sub x13,x12,x10` qua `RegisterFile` → `ALUControl` → `ALU`, rồi kiểm chứng bằng demo (tick 38).

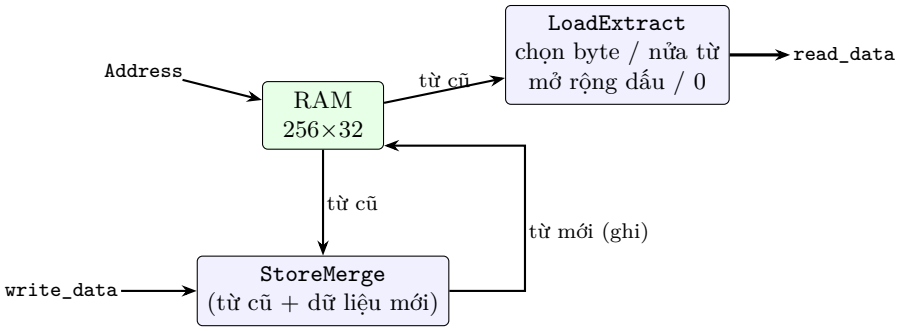
## 6 Giai đoạn 4 — Bộ nhớ và CPU single-cycle (buổi 6–7)

### 6.1 Khối `Memory` cùng `LoadExtract` và `StoreMerge`

- **Mục đích:** bộ nhớ dữ liệu với truy cập byte, nửa từ và từ (`lb lh lw lbu lhu sb sh sw`).
- **Vào / ra:** vào `Address`, `write_data`, `MemWrite`, `CLK`, `funct3`; ra `read_data`.
- **Bên trong:** một RAM 256×32 với *đọc bất đồng bộ* và một chân cho phép ghi (line enables); một splitter địa chỉ (độ lệch byte `Address[1:0]`, chỉ số từ `Address[9:2]`); `LoadExtract`; `StoreMerge`.
- **Vì sao đọc bất đồng bộ:** lệnh load phải có dữ liệu ngay trong chu kỳ. Với đọc đồng bộ mặc định, load trả về dữ liệu không xác định (một trong các lỗi đã gặp).

**`LoadExtract`:** splitter (4 byte, 2 nửa từ), multiplexer 4-ra-1×8 và 2-ra-1×16 chọn byte / nửa từ, 4 bộ mở rộng bit (dấu/0, 8 và 16 → 32), multiplexer 8-ra-1×32 cuối cùng chọn theo `funct3`. `lbu/lhu` mở rộng 0, `lb/lh` mở rộng dấu.

**`StoreMerge`:** RAM chỉ ghi nguyên từ, nên store byte là **đọc–sửa–ghi**: đọc từ cũ, thay một làn byte, ghi cả từ lại. Một ROM 16×4 biến $\{\text{size}, \text{addr}[1:0]\}$ thành bốn tín hiệu cho phép byte (`1 2 4 8` cho byte, `3 3 c c` cho nửa từ, `f` cho từ); bốn multiplexer 2-ra-1 8 bit chọn, theo từng làn, byte mới hay byte cũ; thêm 3 multiplexer byte chọn nguồn byte cho từng làn.




Hình 7: `Memory`: đọc đi qua `LoadExtract`; ghi đi qua `StoreMerge` (đọc–sửa–ghi).

**Thí nghiệm**

Trong `tests/loadextract.vec` và `tests/storemerge.vec`, tìm các dòng `sb` ở độ lệch 2 và `lb` của byte có bit cao bằng 1.

**Câu hỏi tự kiểm tra**

Vì sao `StoreMerge` đọc được từ cũ? Với từ cũ `0x11223344` và $x1 =$ `0xAA`, lệnh `sb x1,1(x0)` ghi gì vào từ 0? (Đáp án: `0x1122AA44`.)

### 6.2 CPU single-cycle `main`

- **Bên trong:** thanh ghi PC (xóa bằng `RESET`); ROM lệnh 256×32 đánh địa chỉ bằng `PC[9:2]`; 2 bộ cộng (`PC+4`, `PC+imm`); cổng AND 32 bit với mặt nạ `0xFFFFFFFE` (cho JALR); một cổng OR (`branch_taken` hoặc `IsJAL`); 5 multiplexer (toán hạng A theo `ASel`, toán hạng B theo `ALUSrc`, write-back theo `WBSel`, hai cái cho PC kế tiếp); 8 sub-circuit.
- **PC kế tiếp:** `next_pc` = `IsJALR` ? `jalr_target` : (`branch_taken` ∨ `IsJAL`) ? `PC+imm` : `PC+4`.
- **Vì sao thiết kế thế:** `lui`, `auipc`, `jal`, `jalr` dùng lại đường ALU và write-back nhờ `ASel` và `WBSel`, không cần phần cứng mới.

**Một lệnh đi qua năm tầng (ví dụ cho `add x10,x10,x11`, tick 3):**

1. **IF:** ROM tại PC = `0x08` xuất `00b50533`; `pc_plus4` = `0x0c`.
2. **ID:** cắt `opcode` = `0x33`, `rd` = 10, `rs1` = 10, `rs2` = 11. `MainControl`: `RegWrite` = 1, `ALUSrc` = 0, `ALUOp` = 10. Register file xuất $x10 = 0$ và $x11 =$ `0x0a`.
3. **EX:** `ALUControl` cho `ALUCtrl` = 0 (ADD); ALU tính $0 +$ `0x0a` = `0x0a`.
4. **MEM:** không có gì.
5. **WB:** `wb_data` = `0x0a` được ghi vào $x10$ ở cạnh xung nhịp; PC = PC+4 = `0x0c`.

**Thí nghiệm**

Dùng các probe ở cuối mạch và bảng ở phần 10: theo dõi một lệnh mỗi lớp (add, addi, bnez, sw, lw, jal, ret) qua năm tầng với giá trị thật.

**Câu hỏi tự kiểm tra**

Vẽ lại `main` từ trí nhớ thành các hộp IF, ID, EX, MEM, WB và đánh dấu ba tín hiệu chọn multiplexer lấy từ đâu. Lệnh nào dùng bộ cộng riêng `PC+imm`? Đích của JALR tính thế nào?

## 7 Giai đoạn 5 — Pipeline (buổi 8–10)

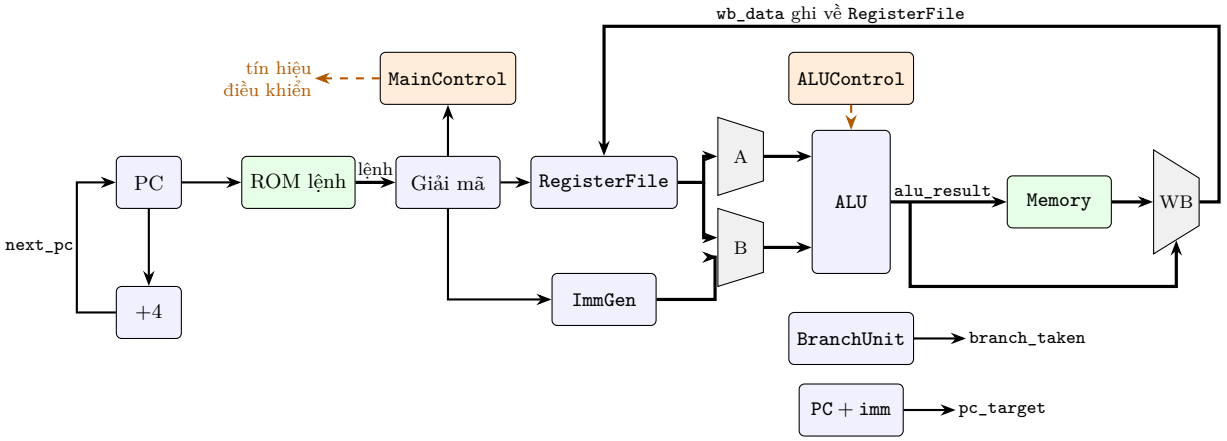
### 7.1 Các tầng, thanh ghi pipeline, thời điểm ghi

Năm tầng làm việc trên năm lệnh cùng lúc. Bốn **thanh ghi pipeline** (IF/ID, ID/EX, EX/MEM, MEM/WB) mang những gì tầng sau cần.

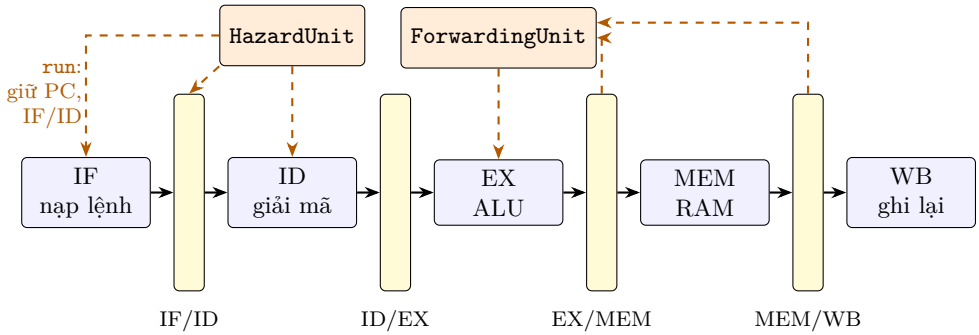
**Bên trong `Pipeline`:** 24 thanh ghi (PC cùng 23 thanh ghi pipeline có nhãn như `pc_id`, `alu_result_mem`); 8 splitter (các bit PC, đóng gói / tách từ điều khiển); 10 sub-circuit; 10 multiplexer (xem phần 9); 3 bộ cộng (`PC+4` ở IF, `pc_ex+imm` và `pc_ex+4` ở EX); hai cổng NOT: một tạo `CLK_n`, một tạo `run` = ¬`stall`; hai cổng OR (`pc_src_ex` và `bubble`).

**Từ điều khiển:** tín hiệu điều khiển được sinh một lần ở ID rồi đóng gói; mỗi tầng chỉ tách phần mình cần.

**Thời điểm ghi:** register file ghi ở **sườn xuống** (`CLK_n`), nên lệnh ở WB ghi trong nửa đầu chu kỳ và lệnh ở ID đọc giá trị mới ở nửa sau: không cần forwarding cho khoảng cách 3 lệnh.




Hình 8: Sơ đồ khối CPU single-cycle (rút gọn): nạp lệnh → giải mã → thực thi → bộ nhớ → ghi lại. Nét cam đứt: tín hiệu điều khiển.




Hình 9: Pipeline 5 tầng, bốn thanh ghi pipeline, hai khối xử lý hazard.

| **Thanh ghi** | **Chứa** |
|---|---|
| IF/ID | `pc_id`, `instr_id` |
| ID/EX | `pc_ex`, `rs1_data_ex`, `rs2_data_ex`, `imm_ex`, `rs1_ex`, `rs2_ex`, `rd_ex`, `funct3_ex`, `funct7_ex`, từ điều khiển `ctrl_ex` (13 bit: `RegWrite ALUSrc MemWrite MemRead IsBranch ALUOp ASel WBSel IsJAL IsJALR`) |
| EX/MEM | `alu_result_mem`, `store_data_mem`, `pc_plus4_mem`, `rd_mem`, `funct3_mem`, điều khiển `RegWrite MemWrite MemRead WBSel` |
| MEM/WB | `alu_result_wb`, `mem_rdata_wb`, `pc_plus4_wb`, `rd_wb`, điều khiển `RegWrite WBSel` |

Bảng 3: Nội dung bốn thanh ghi pipeline (hậu tố `_id`, `_ex`, `_mem`, `_wb` là tầng hiện tại).

**Thí nghiệm**

Mở `Pipeline`, tìm bốn nhóm thanh ghi theo nhãn; với `add x2,x1,x1` hãy ghi lại nội dung từng nhóm sau 1, 2, 3, 4 tick.

**Câu hỏi tự kiểm tra**

Vì sao PC và IF/ID có chân cho phép ghi `run` còn các thanh ghi khác thì không? Vì sao các thanh ghi pipeline bằng 0 lúc đầu và điều đó vô hại?

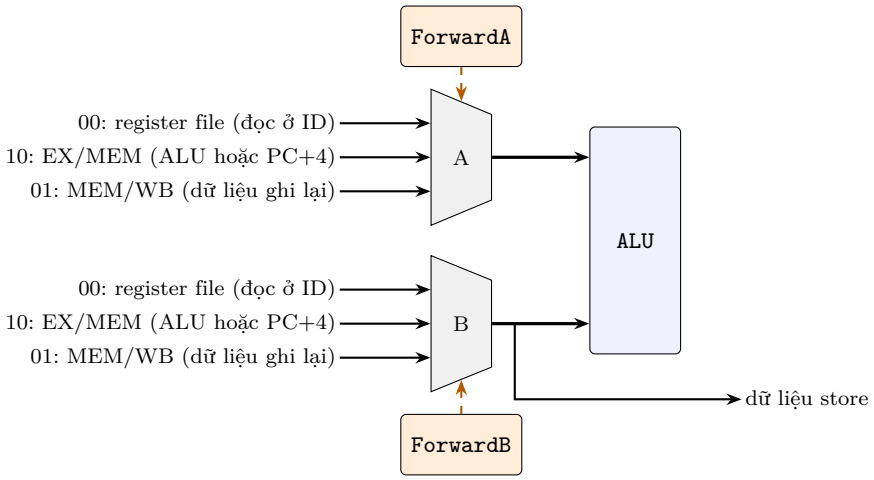
## 7.2 Khối `ForwardingUnit`

- **Mục đích:** đưa cho EX giá trị mới nhất của `rs1`/`rs2` khi lệnh trước chưa kịp ghi.
- **Vào / ra:** vào `rs1_ex`, `rs2_ex`, `rd_mem`, `RegWrite_mem`, `rd_wb`, `RegWrite_wb`; ra `ForwardA`, `ForwardB` (2 bit).
- **Bên trong:** 6 bộ so sánh 5 bit, các cổng AND/NOT, 2 splitter tạo tín hiệu chọn 2 bit.

**Quy tắc:**

$$\texttt{ForwardA} = \begin{cases} 10 & \text{nếu } \texttt{RegWrite_mem} \wedge \texttt{rd_mem} \neq 0 \wedge \texttt{rd_mem} = \texttt{rs1_ex} \quad \text{(từ EX/MEM)} \\ 01 & \text{ngược lại nếu điều tương tự đúng với WB} \quad \text{(từ MEM/WB)} \\ 00 & \text{còn lại} \quad \text{(register file)} \end{cases}$$

EX/MEM được ưu tiên vì mới hơn; điều kiện $\texttt{rd} \neq 0$ ngăn forward `x0`. **Hai chi tiết:** giá trị lấy từ EX/MEM là kết quả ALU *trừ JAL/JALR thì là PC+4*; toán hạng B sau forwarding cũng là dữ liệu của lệnh store.




Hình 10: Forwarding: toán hạng của ALU lấy từ register file (00), MEM/WB (01) hoặc EX/MEM (10).

## 7.3 Khối `HazardUnit`

- **Mục đích:** phát hiện lệnh load theo sau ngay bởi lệnh dùng kết quả của nó.
- **Vào / ra:** vào `MemRead_ex`, `rd_ex`, `rs1_id`, `rs2_id`; ra `stall`.
- **Bên trong:** 3 bộ so sánh 5 bit, cổng OR, NOT và AND 3 ngõ vào.
- **Quy tắc:** $\texttt{stall} = \texttt{MemRead_ex} \wedge \texttt{rd_ex} \neq 0 \wedge (\texttt{rd_ex} = \texttt{rs1_id} \vee \texttt{rd_ex} = \texttt{rs2_id})$.
- **Tác dụng:** $\texttt{run} = \neg\texttt{stall}$ làm PC và IF/ID đứng yên, còn từ điều khiển vào ID/EX bị thay bằng 0 (một bubble).

**Câu hỏi tự kiểm tra**

Vì sao load không forward được như kết quả ALU? Vì sao chỉ stall một chu kỳ mà không phải hai? Bộ so sánh nào bắt điều kiện `rd_ex` = `rs2_id`?

## 7.4 Hazard điều khiển và hiệu năng

Branch, JAL, JALR được quyết định ở **EX**: `pc_src_ex` = `branch_taken` ∨ `IsJAL` ∨ `IsJALR`. PC kế tiếp thành địa chỉ đích, lệnh đang nạp bị thay bằng 0 (`instr_in` = 0) và từ điều khiển vào ID/EX bị xóa: hai lệnh sau lệnh rẽ nhánh bị hủy. Chi phí: 2 chu kỳ cho mỗi branch/lệnh nhảy được rẽ; branch không rẽ không tốn gì (dự đoán tĩnh "không rẽ").

| Tình huống | Chu kỳ thêm |
|---|---|
| lệnh phụ thuộc ngay sau lệnh ALU (forwarding) | 0 |
| dùng giá trị vừa load ở lệnh kế tiếp (load-use) | 1 |
| load, một lệnh không liên quan, rồi mới dùng | 0 |
| branch không rẽ | 0 |
| branch được rẽ / `jal` / `jalr` | 2 |

Tính theo chu kỳ, pipeline ở đây không nhanh hơn single-cycle, vì mô phỏng Logisim không có độ trễ cổng. Lợi ích thật là **chu kỳ xung nhịp ngắn hơn** (đường dài nhất còn một tầng thay vì năm). Cách cải thiện: quyết định JAL ở ID, bộ dự đoán nhánh, bộ đệm đích nhánh.

**Giải thích 64 chu kỳ của `demo.s`:** 39 lệnh + 11 lần chuyển hướng ×2 = 22 (9 `bnez` được rẽ, `jal`, `ret`) = 61; 3 chu kỳ còn lại là thời gian làm đầy pipeline.

### Chương trình `hazards_demo.s` trên pipeline

| Địa chỉ | Lệnh | Mục đích |
|---|---|---|
| 0x00 | `li x1,100` | x1 = 100 |
| 0x04 | `sw x1,0(x0)` | cần x1 ngay: **forwarding** |
| 0x08 | `lw x2,0(x0)` | load |
| 0x0c | `addi x3,x2,1` | dùng x2 ngay: **stall load-use** |
| 0x10 | `add x4,x3,x2` | x3 được forward |
| 0x14 | `beq x4,x4,skip` | luôn rẽ: **flush** |
| 0x18 | `addi x5,x0,1` | không được có hiệu lực |
| 0x1c | `addi x6,x0,2` | không được có hiệu lực |
| 0x20 | `skip: addi x7,x0,7` | lệnh đầu tiên ở đích |
| 0x24 | `halt: j halt` | kết thúc |

Kết quả: $x1 = \texttt{0x64}$, $x2 = \texttt{0x64}$, $x3 = \texttt{0x65}$, $x4 = \texttt{0xc9}$, $x7 = 7$ và $x5 = x6 = 0$.

**Trạng thái các tầng sau mỗi tick (đo trên project)**

| Tick | IF (PC) | ID | EX | MEM | stall | pc_src_ex | FwdA | FwdB |
|---|---|---|---|---|---|---|---|---|
| 1 | sw (04) | li | – | – | 0 | 0 | 00 | 00 |
| 2 | lw (08) | sw | li | – | 0 | 0 | 00 | 00 |
| 3 | addi (0c) | lw | sw | li | 0 | 0 | 00 | **10** |
| 4 | add (10) | addi | lw | sw | **1** | 0 | 00 | 00 |
| 5 | add (**10**, giữ) | addi | bubble | lw | 0 | 0 | 10 | 00 |
| 6 | beq (14) | add | addi | bubble | 0 | 0 | **01** | 00 |
| 7 | addi x5 (18) | beq | add | addi | 0 | 0 | **10** | 00 |
| 8 | addi x6 (1c) | addi x5 | **beq** | add | 0 | **1** | 10 | 10 |
| 9 | skip (**20**) | bubble | bubble | beq | 0 | 0 | 00 | 00 |
| 10 | halt (24) | addi x7 | bubble | bubble | 0 | 0 | 00 | 00 |
| 11 | (28) | halt | addi x7 | bubble | 0 | 0 | 00 | 00 |
| 12 | (2c) | bubble | halt | addi x7 | 0 | **1** | 00 | 00 |

**Thí nghiệm**

Nạp `programs/hazards_demo.hex` vào `Pipeline`, bấm 12 tick và tự điền bảng trên bằng các probe `stall`, `pc_src_ex`, `ForwardA`, `ForwardB` trước khi đọc đáp án.

**Câu hỏi tự kiểm tra**

Vì sao $x5 = x6 = 0$ là bằng chứng flush hoạt động? Nếu không có flush thì sao? Vì sao `ForwardA`=10 ở tick 5 vô hại?

**Bài kiểm tra cuối giai đoạn:** vẽ giản đồ pipeline (IF ID EX ME WB theo từng chu kỳ) của `hazards_demo.s` trên giấy, rồi so với Hình 11.

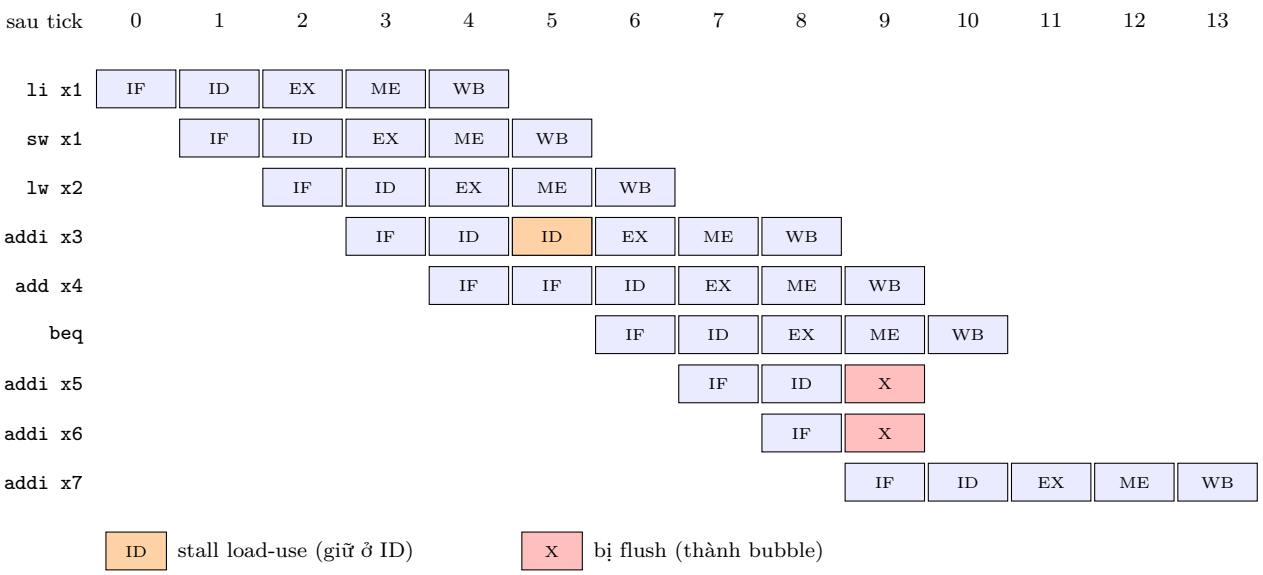
# 8 Giai đoạn 6 — Chương trình, kiểm thử, công cụ (buổi 11–12)

## 8.1 Chương trình và công cụ

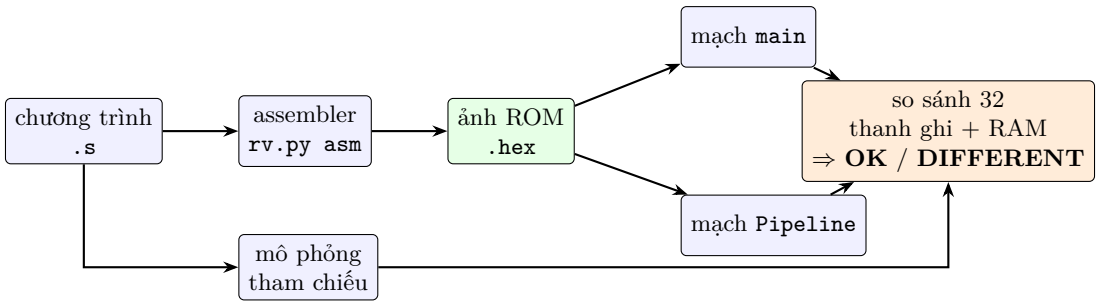
- **Assembler** (`tools/rv.py asm`): nhãn, lệnh giả (`li`, `mv`, `bnez`, `ret`, `j`), macro kiểm tra; tạo ảnh ROM dạng `v2.0 raw`.
- **Bộ mô phỏng tham chiếu** (`tools/rv.py sim`): viết độc lập với mạch; là "chuẩn" của mọi bài test.
- **Bộ chạy không giao diện** (`tools/java/Run.java`): nạp ROM, đảo xung nhịp, đọc PC, thanh ghi và RAM của mạch thật.
- **Bộ sinh mạch** (`tools/build`): sinh circuit từ netlist và kiểm tra từng net.

## 8.2 Ba mức kiểm thử

1. **Test module** (`tests/run_tests.ps1`): 106 hàng test-vector trên 9 module, do chính Logisim chạy.
2. **Chương trình** (`tests/run_cpu_tests.py`): 7 chương trình trên cả hai CPU; sau khi chạy, cả 32 thanh ghi và mọi từ dữ liệu khác 0 phải bằng kết quả của bộ mô phỏng tham chiếu.
3. **Chương trình ngẫu nhiên** (`tests/fuzz.py`): chương trình ngẫu nhiên 60–80 lệnh chỉ dùng 7 thanh ghi (nên hầu hết lệnh đều phụ thuộc lệnh gần đó).




Hình 11: Pipeline của `hazards_demo.s`. Cột là trạng thái sau mỗi tick. Cam: lệnh `addi x3` bị giữ ở ID một chu kỳ (stall load-use). Đỏ (X): hai lệnh sau `beq` bị flush thành bubble.




Hình 12: Cách chứng minh: cùng một chương trình chạy trên hai mạch và trên bộ mô phỏng độc lập, rồi so sánh.

**Thí nghiệm**

Viết `test1.s` (giai thừa của 5), chạy `python tools/live_proof.py test1.s`; sau đó sửa một lệnh và *dự đoán* kết quả trước khi chạy.

**Câu hỏi tự kiểm tra**

Vì sao bộ mô phỏng tham chiếu phải độc lập với mạch? Bộ fuzz tìm ra điều gì mà test viết tay không tìm?

### 8.3 Lỗi đã tìm thấy và bài học

| Ở đâu | Lỗi | Cách sửa |
|---|---|---|
| ALU | `SLTU` trả `FFFFFFFF` thay vì 1 (mở rộng dấu) | mở rộng 0 |
| ALUControl | `SLL` kiểu R chọn mã của `SRL` | sửa hằng số |
| MainControl | JALR so với opcode của JAL; LUI chọn hằng số kiểu B | viết lại phần giải mã |
| ImmGen | kiểu U thiếu $\ll 12$; bit 0 kiểu J bằng 1 | đã sửa |
| Memory | load trả dữ liệu không xác định (RAM đọc đồng bộ) | đọc bất đồng bộ |
| Datapath | JAL, JALR, LUI, AUIPC chưa có đường dữ liệu | thêm `ASel`, `WBSel`, logic PC |

**Cạm bẫy của Logisim:** dây chỉ nối ở *đầu mút* (không nối ở chỗ giao chữ T); mọi thứ phải nằm trên lưới 10 px; `Constant` mặc định bằng 1; RAM mặc định đọc đồng bộ.

## 9 Đọc sâu file mạch `RiscVComputer.circ`

Phần này lấy trực tiếp từ file `.circ` (Logisim Evolution 4.1.0): số linh kiện thật, cách các dây và tunnel nối với nhau, và những chi tiết chỉ thấy khi lần từng net. Nên đọc sau giai đoạn 5.

## 9.1 Số linh kiện trong từng circuit

| Circuit | Linh kiện (không kể Pin, Tunnel, Text, Probe) |
|---|---|
| `InstructionDecode` | 1 splitter (32 → 6 trường) |
| `MainControl` | 9 bộ so sánh 7 bit + 9 hằng số opcode; 11 cổng OR (8 cổng 3 ngõ vào, 3 cổng 2 ngõ vào); 4 splitter ghép bus |
| `ImmGen` | 9 splitter; 4 bộ mở rộng bit; 3 hằng số (hai 1 bit, một 12 bit); 1 MUX 8:1 |
| `RegisterFile` | 32 thanh ghi; 1 DEMUX 1:32; 2 MUX 32:1; 1 hằng số 0 |
| `ALUControl` | 19 hằng số 4 bit; 5 MUX (hai 8:1, một 4:1, hai 2:1); 1 splitter tách `funct7[5]` |
| `ALU` | bộ cộng, bộ trừ, AND/OR/XOR 32 bit, 3 bộ dịch, 2 bộ so sánh, 2 bộ mở rộng 0, 1 MUX 16:1, 1 splitter lấy `B[4:0]` |
| `BranchUnit` | 2 bộ so sánh 32 bit; 3 NOT; 1 MUX 8:1 một bit; 1 AND; 1 hằng số 0 |
| `Memory` | 1 RAM 256×32 (`asyncread=true`); 1 splitter địa chỉ; `LoadExtract`; `StoreMerge` |
| `LoadExtract` | 3 splitter; 3 MUX (4:1×8, 2:1×16, 8:1×32); 4 bộ mở rộng bit |
| `StoreMerge` | 6 splitter; 1 ROM 16×4; 7 MUX (ba 4:1×8 chọn nguồn byte, bốn 2:1×8 chọn byte mới/cũ) |
| `ForwardingUnit` | 6 bộ so sánh 5 bit; 4 NOT; 8 AND 2 ngõ vào; 2 splitter; 1 hằng số 0 |
| `HazardUnit` | 3 bộ so sánh 5 bit; 1 NOT; 1 OR; 1 AND 3 ngõ vào; 1 hằng số 0 |
| `main` | thanh ghi PC; ROM 256×32; 2 bộ cộng; 5 MUX; 1 AND 32 bit; 1 OR; 4 hằng số; 8 sub-circuit; 5 probe |
| `Pipeline` | 24 thanh ghi; ROM; 3 bộ cộng; 10 MUX; 2 NOT; 2 OR; 1 AND; 8 hằng số; 8 splitter; 10 sub-circuit; 8 probe |

ROM lệnh của cả hai CPU đang nạp sẵn `demo.s` (13 từ, từ `00000513` đến `00008067`). Muốn chạy `hazards_demo.s` phải nạp file `.hex` khác vào ROM.

## 9.2 Phương trình đầy đủ của `MainControl`

Lần theo dây vào từng cổng OR cho kết quả sau (bảy lớp lệnh ghi `rd` được gom bằng hai cổng OR trung gian `rw_a`, `rw_b`):

$$\texttt{RegWrite} = \underbrace{(\texttt{IsR} \vee \texttt{IsI} \vee \texttt{IsLoad})}_{\texttt{rw_a}} \vee \underbrace{(\texttt{IsLui} \vee \texttt{IsAuipc} \vee \texttt{IsJAL})}_{\texttt{rw_b}} \vee \texttt{IsJALR}$$

$$\texttt{ALUSrc} = \underbrace{(\texttt{IsI} \vee \texttt{IsLoad} \vee \texttt{IsStore})}_{\texttt{as_a}} \vee \underbrace{(\texttt{IsLui} \vee \texttt{IsAuipc} \vee \texttt{IsJALR})}_{\texttt{as_b}}$$

$$\texttt{MemRead} = \texttt{IsLoad}, \qquad \texttt{MemWrite} = \texttt{IsStore} \quad \text{(nối thẳng, không qua cổng)}$$

- **LUI** có `ALUSrc` = 1 và `ASel` = 10: ALU tính 0 + `imm`, nên kết quả chính là hằng số đã dịch trái 12 bit.
- **JAL** có `ALUSrc` = 0: ALU vẫn tính `rs1+rs2` nhưng kết quả bị bỏ, vì `WBSel`=10 ghi PC+4 và đích nhảy lấy từ bộ cộng `PC+imm`.
- Bộ so sánh trong `MainControl` đặt chế độ *unsigned*, nhưng chỉ dùng ngõ ra "bằng", nên chế độ không ảnh hưởng.

### 9.3 Bảng mã trong `ALUControl`

| `funct3` | **MUX kiểu R** | **MUX kiểu I** |
|---|---|---|
| 000 | MUX ADD/SUB (theo `funct7[5]`) | hằng 0 (ADDI) |
| 001 | 5 (SLL) | 5 (SLLI) |
| 010 | 8 (SLT) | 8 (SLTI) |
| 011 | 9 (SLTU) | 9 (SLTIU) |
| 100 | 4 (XOR) | 4 (XORI) |
| 101 | MUX SRL/SRA (dùng chung) | MUX SRL/SRA (dùng chung) |
| 110 | 3 (OR) | 3 (ORI) |
| 111 | 2 (AND) | 2 (ANDI) |

- MUX SRL/SRA có một ngõ ra nối vào *cả hai* bảng, vì `srai` và `sra` đều phân biệt bằng bit 30 của lệnh (`funct7[5]`).
- Kiểu I với `funct3`=000 dùng thẳng hằng số 0, *không* qua MUX ADD/SUB. Ở `addi` bit 30 là bit 10 của hằng số, nên `addi x1,x0,-1024` (bit 30 = 1) vẫn là phép cộng. Đây là chỗ dễ sai nhất của bảng này.
- MUX cuối chọn theo `ALUOp`: ngõ 0 hằng 0 (ép ADD), ngõ 1 hằng 1 (ép SUB), ngõ 2 bảng R, ngõ 3 bảng I.

### 9.4 Mười multiplexer của `Pipeline`

| **Ra** | **Chọn theo** | **Các ngõ vào (0, 1, 2, 3)** |
|---|---|---|
| `next_pc` | `pc_src_ex` | `pc_plus4_if`, `target_ex` |
| `instr_in` | `pc_src_ex` | lệnh từ ROM, 0 (flush lệnh vừa nạp) |
| `ctrl_in` (13 bit) | `bubble` | `ctrl_id`, 0 (bubble) |
| `fwd_mem_val` | `WBSel1_mem` | `alu_result_mem`, `pc_plus4_mem` |
| `opA` | `ForwardA` | `rs1_data_ex`, `wb_data`, `fwd_mem_val`, `rs1_data_ex` |
| `opB` | `ForwardB` | `rs2_data_ex`, `wb_data`, `fwd_mem_val`, `rs2_data_ex` |
| `alu_a` | `ASel_ex` | `opA`, `pc_ex`, 0, 0 |
| `alu_b` | `ALUSrc_ex` | `opB`, `imm_ex` |
| `target_ex` | `IsJALR_ex` | `pc_target` (`pc_ex+imm`), `jalr_target` (ALU AND `0xFFFFFFFE`) |
| `wb_data` | `WBSel_wb` | `alu_result_wb`, `mem_rdata_wb`, `pc_plus4_wb`, 0 |

- **Mã không dùng vẫn an toàn:** ngõ 3 của MUX forwarding quay về giá trị register file; ngõ 3 của `ASel` và `WBSel` là 0.
- **Forwarding nằm trước ALUSrc:** `opB` được forward trước rồi mới chọn giữa `opB` và `imm_ex`. Lệnh kiểu I có thể bật `ForwardB` vì trường `rs2` thực chất là bit hằng số, nhưng vô hại vì `ALUSrc`=1 bỏ qua `opB`. Còn `opB` thì đồng thời là `store_data_mem`, nên lệnh store nhận được dữ liệu đã forward.
- **BranchUnit và JALR dùng toán hạng đã forward:** `BranchUnit` nhận `opA`/`opB`, còn đích JALR lấy từ ALU. Vì vậy `beq` hay `ret` đứng ngay sau lệnh ALU ghi thanh ghi nguồn đều không phải stall.
- **PC+4 được tính lại ở EX:** ID/EX không chứa PC+4. Bộ cộng thứ ba tính `pc_ex+4` rồi đưa vào `pc_plus4_mem` làm địa chỉ trả về của JAL/JALR.
- **`bubble` = `stall` OR `pc_src_ex`.** Hai tín hiệu không bao giờ cùng bằng 1, vì `stall` cần một lệnh load ở EX còn `pc_src_ex` cần một lệnh nhảy ở EX, mà EX chỉ chứa một lệnh.

## 9.5 Đóng gói từ điều khiển

Splitter 13 bit ở ID và EX chia từ điều khiển thành 10 trường theo thứ tự bit 0 → 12: `RegWrite`(0), `ALUSrc`(1), `MemWrite`(2), `MemRead`(3), `IsBranch`(4), `ALUOp`(5–6), `ASel`(7–8), `WBSel`(9–10), `IsJAL`(11), `IsJALR`(12). Sang EX/MEM còn 5 bit (`RegWrite`, `MemWrite`, `MemRead`, `WBSel`); sang MEM/WB còn 3 bit (`RegWrite`, `WBSel`).

## 9.6 Xung nhịp

- PC, 23 thanh ghi pipeline và RAM dữ liệu ghi ở **sườn lên** của `CLK`.
- `RegisterFile` trong `Pipeline` nhận `CLK_n` (qua cổng NOT) nên ghi ở **sườn xuống**, giữa chu kỳ.
- Trong `main` chỉ có một miền xung nhịp: register file, PC và RAM cùng ghi ở sườn lên.

## 9.7 Những điểm mạch *không* làm (nên biết khi bị hỏi)

- **`MemRead` không điều khiển bộ nhớ.** `Memory` không có chân `MemRead`; RAM luôn đọc bất đồng bộ. Trong `main`, tunnel `MemRead` chỉ có một đầu (không nối đi đâu). Trong `Pipeline`, chỉ `MemRead_ex` được dùng (vào `HazardUnit`); `MemRead_mem` được mang theo nhưng không có nơi nhận.
- **`HazardUnit` stall thừa.** Mạch so `rd_ex` với `rs1_id` và `rs2_id` mà không xét lệnh có thật sự dùng hai thanh ghi đó không. Ví dụ `lw x2,0(x0)` rồi `addi x3,x5,2`: bit 24:20 của `addi` là hằng số 2, trùng `rd_ex`=2, nên vẫn stall 1 chu kỳ dù không cần. Kết quả vẫn đúng, chỉ chậm hơn. Muốn sửa: thêm tín hiệu "dùng `rs2`" (chỉ kiểu R, S, B) và "dùng `rs1`" (mọi lệnh trừ LUI, AUIPC, JAL).
- **Không bẫy truy cập lệch (misaligned).** `lw` bỏ qua `Address[1:0]` và đọc nguyên từ; `lh` chỉ xét `Address[1]`. Trong ROM của `StoreMerge` (`1 2 4 8 3 3 c c f f f f f f f f`), `sh` ở độ lệch 1 ghi như độ lệch 0, ở độ lệch 3 ghi như độ lệch 2; `funct3` có size 11 được coi như ghi từ.
- **Địa chỉ bị lặp mỗi 1 KB.** RAM chỉ dùng `Address[9:2]`, bit 31:10 bị bỏ, nên `0x400` trỏ về cùng từ 0 như `0x000`. ROM lệnh cũng chỉ dùng `PC[9:2]`: PC vượt `0x3FC` sẽ quay về đầu chương trình.
- **Opcode lạ thành NOP.** Mọi cờ lớp lệnh bằng 0 nên không ghi thanh ghi, không ghi bộ nhớ, không nhảy. Mạch không báo lỗi lệnh không hợp lệ.

**Câu hỏi tự kiểm tra**

- Vì sao `addi x1,x0,-1024` không bị biến thành phép trừ?
- Trong `Pipeline`, `addi x1,x0,5` rồi ngay sau là `beq x1,x0,L`: giá trị x1 đến `BranchUnit` bằng đường nào?
- Viết một cặp lệnh làm `HazardUnit` stall dù không có phụ thuộc thật.
- Nếu xoá tunnel `MemRead` trong `main`, mạch có chạy khác đi không?

# 10 Demo từng bước

## 10.1 Chương trình `demo.s`

| Địa chỉ | Mã máy | Lệnh | Ý nghĩa |
|---|---|---|---|
| 0x00 | `00000513` | `li x10,0` | tổng = 0 |
| 0x04 | `00a00593` | `li x11,10` | biến đếm = 10 |
| 0x08 | `00b50533` | `loop: add x10,x10,x11` | tổng += biến đếm |
| 0x0c | `fff58593` | `addi x11,x11,-1` | biến đếm − = 1 |
| 0x10 | `fe059ce3` | `bnez x11,loop` | lặp nếu khác 0 |
| 0x14 | `00a02023` | `sw x10,0(x0)` | lưu tổng |
| 0x18 | `00002603` | `lw x12,0(x0)` | nạp lại vào x12 |
| 0x1c | `010000ef` | `jal x1,double` | gọi hàm, x1 = 0x20 |
| 0x20 | `40a606b3` | `sub x13,x12,x10` | 110 − 55 = 55 |
| 0x24 | `0646a713` | `slti x14,x13,100` | 55 < 100 ⇒ 1 |
| 0x28 | `0000006f` | `halt: j halt` | vòng lặp kết thúc |
| 0x2c | `00c60633` | `double: add x12,x12,x12` | x12 = 110 |
| 0x30 | `00008067` | `ret` | về địa chỉ trong x1 |

Kết quả: tổng $10 + 9 + \cdots + 1 = 55 =$ `0x37`; sau lời gọi $x12 = 110 =$ `0x6e`; $x13 =$ `0x37`; $x14 = 1$; từ dữ liệu 0 = `0x37`; PC cuối = `0x28`; $x1 =$ `0x20`.

## 10.2 Cách đọc một tick

- Một lần Ctrl+F9 = một chu kỳ xung nhịp = một lệnh (single-cycle).
- Sau tick, PC đã trỏ tới lệnh *kế tiếp*. Hàng probe hiện lệnh kế tiếp (`instr`) và nó *sẽ* tính ra gì; tab State hiện các thanh ghi mà lệnh *trước* vừa ghi.
- Nhịp độ: nói một câu, bấm một tick, chỉ một thứ, chờ hai giây.

## 10.3 Một lệnh đi qua CPU, theo từng nhóm

**Kiểu R: `add x10,x10,x11` (tick 3)**
ID: `ALUOp` = 10, `ALUSrc` = 0; EX: `ALUCtrl` = 0, kết quả `0x0a`; WB: ghi vào $x10$.

**Kiểu I: `addi x11,x11,-1` (tick 4)**
hằng số `0xffffffff`; `ALUSrc` = 1; `ALUOp` = 11; kết quả `0x09`.

**Branch rẽ: `bnez x11,loop` (tick 5)**
hằng số kiểu B = −8; `BranchUnit` so $x11 = 9$ với $x0 = 0$ bằng `funct3` = 001 (BNE): `branch_taken` = 1; `pc_target` = `0x10` − 8 = `0x08`.

**Store: `sw x10,0(x0)` (tick 33)**
`MemWrite` = 1; địa chỉ 0; `StoreMerge` bật cả bốn byte; từ 0 của RAM thành `0x37`.

**Load: `lw x12,0(x0)` (tick 34)**
`LoadExtract` cho qua nguyên từ; `WBSel` = 01; $x12 =$ `0x37`.

**JAL: `jal x1,double` (tick 35)**
`pc_target` = `0x1c` + `0x10` = `0x2c`; `WBSel` = 10: $x1 =$ `0x20`.

**JALR: `ret` (tick 37)**
ALU tính $x1 + 0 =$ `0x20`; xóa bit 0; `next_pc` = `0x20`; `rd` = $x0$ nên không ghi gì.

**SUB và SLTI (tick 38, 39)**
`funct7` = 0100000 chọn SUB: `0x6e` − `0x37` = `0x37`; `slti`: `0x37` < 100 ⇒ $x14 = 1$.

**Kết thúc: `halt: j halt`**
nhảy về chính nó, PC đứng ở `0x28`.

### 10.4 Cả 40 tick của `demo.s` (single-cycle)

Tick $n$ thực thi lệnh ở PC *trước* tick; hai cột cuối là trạng thái *sau* tick.

| Tick | PC chạy | Lệnh | Thay đổi | PC sau |
|---|---|---|---|---|
| 1 | 0x00 | `li x10,0` | (x10 vẫn 0) | 0x04 |
| 2 | 0x04 | `li x11,10` | x11 = 0x0a | 0x08 |
| 3 | 0x08 | `add` | x10 = 0x0a | 0x0c |
| 4 | 0x0c | `addi` | x11 = 0x09 | 0x10 |
| 5 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 6 | 0x08 | `add` | x10 = 0x13 | 0x0c |
| 7 | 0x0c | `addi` | x11 = 0x08 | 0x10 |
| 8 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 9 | 0x08 | `add` | x10 = 0x1b | 0x0c |
| 10 | 0x0c | `addi` | x11 = 0x07 | 0x10 |
| 11 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 12 | 0x08 | `add` | x10 = 0x22 | 0x0c |
| 13 | 0x0c | `addi` | x11 = 0x06 | 0x10 |
| 14 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 15 | 0x08 | `add` | x10 = 0x28 | 0x0c |
| 16 | 0x0c | `addi` | x11 = 0x05 | 0x10 |
| 17 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 18 | 0x08 | `add` | x10 = 0x2d | 0x0c |
| 19 | 0x0c | `addi` | x11 = 0x04 | 0x10 |
| 20 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 21 | 0x08 | `add` | x10 = 0x31 | 0x0c |
| 22 | 0x0c | `addi` | x11 = 0x03 | 0x10 |
| 23 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 24 | 0x08 | `add` | x10 = 0x34 | 0x0c |
| 25 | 0x0c | `addi` | x11 = 0x02 | 0x10 |
| 26 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 27 | 0x08 | `add` | x10 = 0x36 | 0x0c |
| 28 | 0x0c | `addi` | x11 = 0x01 | 0x10 |
| 29 | 0x10 | `bnez` (rẽ) | – | 0x08 |
| 30 | 0x08 | `add` | x10 = 0x37 (= 55) | 0x0c |
| 31 | 0x0c | `addi` | x11 = 0x00 | 0x10 |
| 32 | 0x10 | `bnez` (**không** rẽ) | – | 0x14 |
| 33 | 0x14 | `sw` | từ 0 = 0x37 | 0x18 |
| 34 | 0x18 | `lw` | x12 = 0x37 | 0x1c |
| 35 | 0x1c | `jal` | x1 = 0x20 | **0x2c** |
| 36 | 0x2c | `add x12,x12,x12` | x12 = 0x6e (= 110) | 0x30 |
| 37 | 0x30 | `ret` | – | **0x20** |
| 38 | 0x20 | `sub` | x13 = 0x37 | 0x24 |
| 39 | 0x24 | `slti` | x14 = 0x01 | 0x28 |
| 40 | 0x28 | `halt` | – | 0x28 (đứng yên) |

**Vì sao kết quả ghi 39 chu kỳ chứ không phải 40:** tick 40 chỉ lặp lại lệnh nhảy cuối; lần ghi thanh ghi cuối cùng xảy ra ở tick 39. **Các điểm nên chỉ:** PC *quay lùi* ở tick 5, 8, 11, ... (vòng lặp); ở tick 32 nó *đi tiếp*; ở tick 35 nó *nhảy tới* `0x2c`; ở tick 37 nó *trở về* `0x20`.

### 10.5 Các giá trị probe cần xem (single-cycle)

Giá trị hiện ở hàng probe *sau* tick, tức của lệnh ở PC mới:

| Sau tick | PC | instr | alu_result | branch_taken | wb_data |
|---|---|---|---|---|---|
| 1 | 04 | 00a00593 | 0a | 0 | 0a |
| 2 | 08 | 00b50533 | 0a | 0 | 0a |
| 3 | 0c | fff58593 | 09 | 0 | 09 |
| 4 | 10 | fe059ce3 | 09 | **1** | 09 |
| 5 | **08** | 00b50533 | 13 | 0 | 13 |
| 31 | 10 | fe059ce3 | 00 | **0** | 00 |

## 11 Chia việc cho nhóm ba người

| Người | Chủ đề | Circuit | Bước demo |
|---|---|---|---|
| A | front end, giải mã, single-cycle | `main`, `InstructionDecode`, `MainControl`, `ImmGen`, `RegisterFile` | 1–5 |
| B | thực thi và bộ nhớ, kiểm thử | `ALU`, `ALUControl`, `BranchUnit`, `Memory`, `LoadExtract`, `StoreMerge` | 6, 9, 11, 12 |
| C | pipeline và hazard | `Pipeline`, `ForwardingUnit`, `HazardUnit` | 7, 8, 10 |

Cả 14 circuit đều có người phụ trách: A có 5, B có 6, C có 3 (nhưng `Pipeline` là circuit lớn nhất).

## 12 Tự đánh giá

| Mức | Bạn có thể . . . |
|---|---|
| 1 | gọi tên mọi circuit và nói nó làm gì trong một câu |
| 2 | liệt kê các linh kiện bên trong mọi circuit và nói vì sao có chúng |
| 3 | lần theo bất kỳ lệnh nào qua `main` với giá trị thật |
| 4 | dự đoán hành vi pipeline (stall, forwarding, flush) của một chương trình ngắn |
| 5 | giải thích dự án được kiểm thử thế nào và vì sao tin được kết quả |

Mục tiêu: mức 5 cho phần của riêng bạn và mức 3 cho hai phần còn lại.

**Bài thi cuối (không xem tài liệu):** (1) vẽ `main` và `Pipeline`; (2) lần theo `lw` theo sau bởi một `addi` phụ thuộc qua pipeline; (3) giải thích ba lỗi đã gặp; (4) giải thích vì sao có thể tin phần chứng minh.

**Thứ tự đọc gợi ý:** `README.md` → `docs/GUIDE.md` → `docs/REPORT.md` → `docs/COMPONENTS.md` → `docs/DEMO_EXPLAIN.md` → `docs/PRESENTATION.md` → `docs/TOOLS.md`, `docs/DESIGN_LOG.md`.